CS7IS2: Artificial Intelligence Assignment 1

Faith Olopade - 21364066

Introduction

This report presents a comparative analysis of search based and Markov Decision Process (MDP) algorithms for maze solving. Maze solving is a canonical problem in artificial intelligence that provides a controlled environment for comparing algorithmic paradigms, classical graph search, which seeks a single solution path, and sequential decision making under the MDP framework, which computes optimal policies for entire state spaces.

Four search algorithms are implemented and evaluated, Depth First Search (DFS), Breadth First Search (BFS), A* with Manhattan distance heuristic, and A* with Euclidean distance heuristic. Two MDP algorithms are also implemented, Value Iteration and Policy Iteration. These six algorithms are compared across mazes of five different sizes (5×5, 10×10, 20×20, 30×30, and 50×50 cells), using multiple performance metrics including execution time, nodes explored, path length, maximum frontier size, and iterations to convergence.

Beyond the core comparison, this report includes a sensitivity analysis examining how MDP specific parameters, the discount factor γ and the transition stochasticity p_intended, affect convergence behaviour and solution quality. All experiments are repeated three times per configuration to enable statistical analysis.

Implementation Design Choices

Maze Generator

The maze generator uses the Recursive Backtracking algorithm (randomised DFS) to produce perfect mazes, mazes in which there exists exactly one path between any two cells. This property is important for fair algorithm comparison, since perfect mazes have a unique solution, all correct algorithms must find paths of identical length when they find optimal paths.

The maze is represented as a 2D NumPy array (grid) of size (2N+1) × (2N+1) for an N×N cell maze, where cells occupy odd indexed positions and walls occupy even indexed positions. A cell value of 0 represents a passage and 1 represents a wall. The start position is fixed at grid position (1,1) (top left cell) and the goal at (2N−1, 2N−1) (bottom right cell). Seeds are used for reproducibility so that the same maze is generated for all algorithms within a given run.

The maze object provides a get_neighbours(row, col) method that returns passable adjacent cells by checking whether the wall between two cells has been removed. This interface is shared by both search and MDP algorithms. Figure 1 shows example solved mazes at 5×5 scale, demonstrating the perfect maze structure with green start, red goal, and blue solution path.

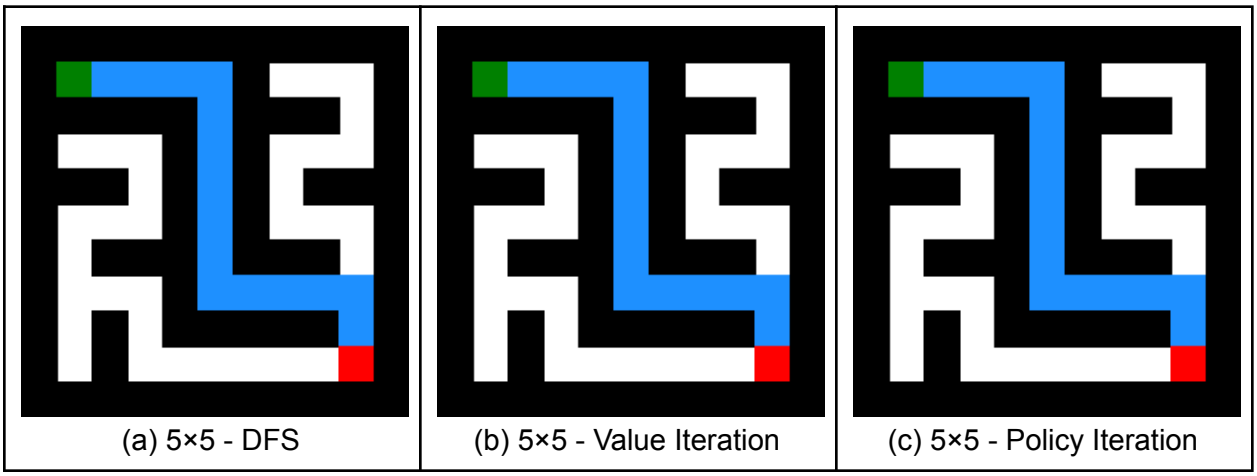

Figure 1: Solved 5×5 mazes showing green start (top left), red goal (bottom right), and blue solution path. All algorithms find the same path in perfect mazes.

Search Algorithms

DFS is implemented iteratively using an explicit stack to avoid Python's recursion limit on large mazes. It expands the most recently discovered node first, meaning it goes deep into a single branch before backtracking. DFS is not guaranteed to find the shortest path, however, in perfect mazes where only one path exists, all search algorithms find the same solution. DFS has O(V+E) time complexity and maintains a frontier proportional to the longest path depth, making it the most memory efficient search algorithm tested.

BFS is implemented using a FIFO queue (collections.deque). It expands nodes level by level, guaranteeing the shortest path in terms of number of steps. BFS has O(V+E) time complexity but maintains a frontier proportional to the branching factor raised to the depth, resulting in higher peak memory usage than DFS.

A* with Manhattan Distance uses f(n) = g(n) + h(n) where g(n) is the cost so far and h(n) = |row_n − row_goal| + |col_n − col_goal|. This is admissible (never overestimates) and consistent for 4 connected grid movement, guaranteeing optimal solutions. A* is implemented with a min heap priority queue and a closed set to avoid re-expansion.

A* with Euclidean Distance uses the same framework but with h(n) = √((row_n − row_goal)2 + (col_n − col_goal)^2). The Euclidean heuristic is also admissible and consistent, but provides a looser lower bound than Manhattan distance on 4 connected grids because the actual shortest

path must follow cardinal directions. As a consequence, A* Euclidean typically explores slightly more nodes than A*Manhattan.

MDP Algorithms

The maze is formulated as an MDP with the following components:

- States (S) every passable cell in the grid (approximately N2 states for an N×N cell maze)
- Actions (A) four cardinal directions (up, down, left, right), where moving into a wall keeps the agent in place
- Transition function T(s, a, s') parameterised by p_intended (with p_intended=1.0, transitions are deterministic with p_intended < 1.0, there is a probability of (1−p)/2 of slipping to each perpendicular direction)
- Rewards R(s, a, s') step reward of −1.0 for every transition and goal reward of +100.0, Discount factor γ = 0.99 by default; Convergence threshold $\theta = 10^{-6}$.

Value Iteration applies the Bellman optimality equation iteratively: V(s) = max_a Σ_s' T(s,a,s')[R(s,a,s') + γ V(s')]. It sweeps over all states each iteration, updating values in place, until the maximum change falls below θ. Total state action evaluations (|S| × |A| per sweep) are tracked as the primary work metric.

Policy Iteration alternates between:

- Policy Evaluation computing Vπ for the current policy iteratively until convergence and
- Policy Improvement deriving a greedy policy from the current value function.

The algorithm terminates when the policy is stable (no changes in a full improvement sweep). Each outer iteration is more expensive than a VI sweep because policy evaluation itself iterates until convergence. The max_iterations cap is set to 1000.

Choice of Comparison Metrics

- **Execution Time (seconds):** Wall clock time via time.perf_counter(). The most practical metric but hardware dependent.
- **Nodes Explored / State Action Evaluations:** For search, nodes expanded, for MDP, total state action evaluations across all iterations. Hardware independent work measure.
- **Path Length (grid steps):** Solution quality number of cells in the path from start to goal.
- **Maximum Frontier Size:** Peak memory for search algorithms (peak open set size).
- **Iterations to Convergence:** Outer loop count for MDP algorithms. These metrics span time complexity, space complexity, and solution quality as recommended by Russell and Norvig (2021).

Experimental Setup

All experiments were run on a Windows machine with Python 3.12. The experimental configuration is summarised in Table 1.

Parameter	Value
Maze sizes	5×5, 10×10, 20×20, 30×30, 50×50
Runs per size	3
Maze generator	Recursive Backtracking (seeded)
Search algorithms	DFS, BFS, A* Manhattan, A* Euclidean
MDP algorithms	Value Iteration, Policy Iteration
γ (default)	0.99
θ	$1e^{-6}$
p_intended (default)	1.0 (deterministic)
Step / Goal reward	−1.0 / +100.0
Max iterations	1000

Table 1: Experimental Parameters

For the sensitivity analysis, γ was varied across {0.9, 0.95, 0.99} and p_intended across {0.7, 0.8, 0.9, 1.0} on maze sizes {5, 10, 20} with 3 runs each. Each maze was generated with a deterministic seed (size × 1000 + run_number) ensuring identical mazes across algorithms within a run, but different mazes across runs to capture variability.

Results and Performance Analysis

Search Algorithm Comparison

All four search algorithms achieved a 100% success rate across all maze sizes and runs. Since the mazes are perfect (exactly one path between any two cells), all algorithms found paths of identical length for a given maze instance. Figure 2 shows solved mazes at 10×10 scale, illustrating how BFS and A* Euclidean find identical paths.

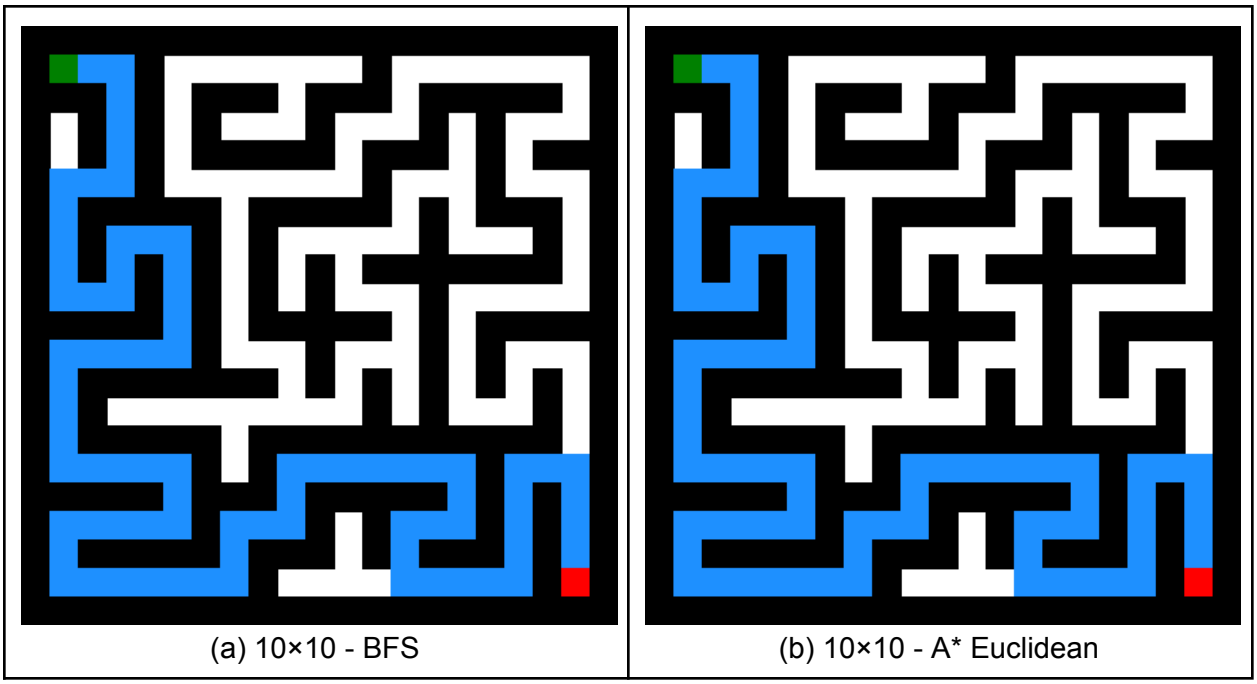

Figure 2: Solved 10×10 mazes. Both algorithms find identical paths in perfect mazes.

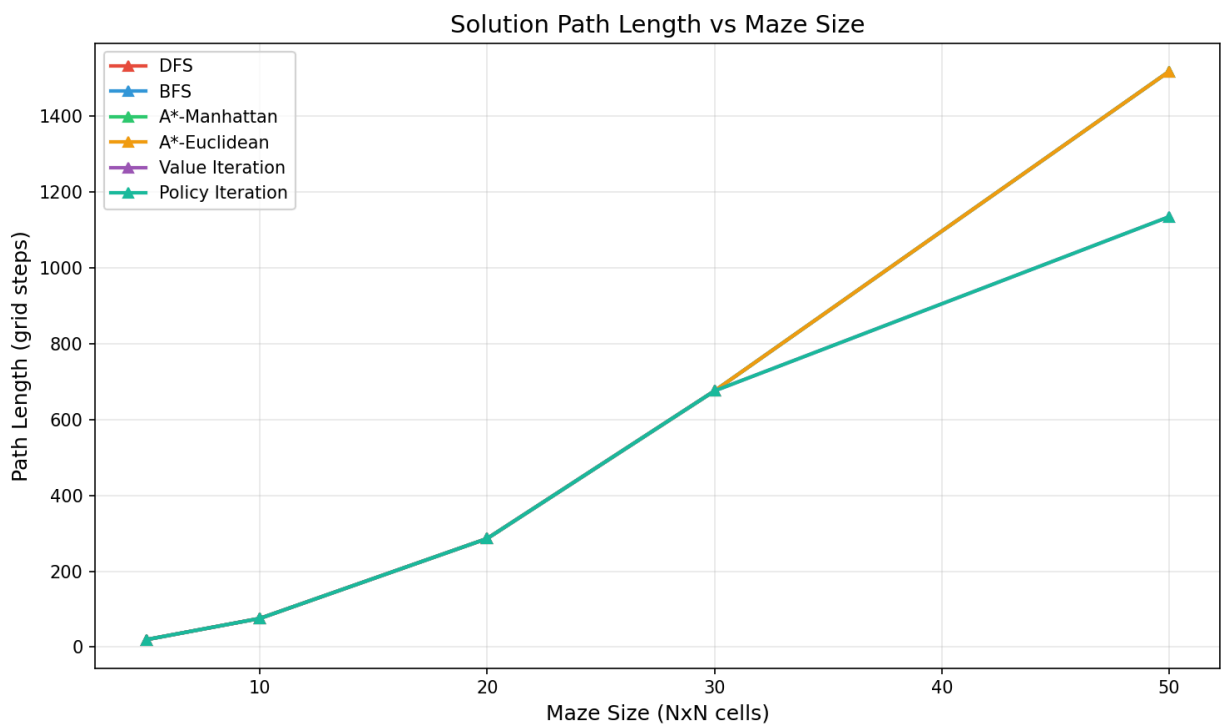

Figure 3: Solution Path Length vs Maze Size

Path lengths scale roughly quadratically with maze size from a mean of 19.7 steps at 5×5 to 1,518.3 steps at 50×50. The identical path lengths across all six algorithms confirm the unique solution property and validate the correctness of all implementations.

Nodes Explored: DFS consistently explored the fewest nodes across all sizes, reaching a mean of 1,718 at 50×50, compared to BFS at 2,849 and A* Manhattan/Euclidean at approximately 2,686 - 2,719. DFS explores fewer nodes because it follows a single path without systematically exploring alternatives. BFS explored the most because it expands all nodes at each depth level. A* Manhattan and A* Euclidean explored nearly identical counts, with Manhattan marginally fewer (2,686 vs 2,719 at 50×50).

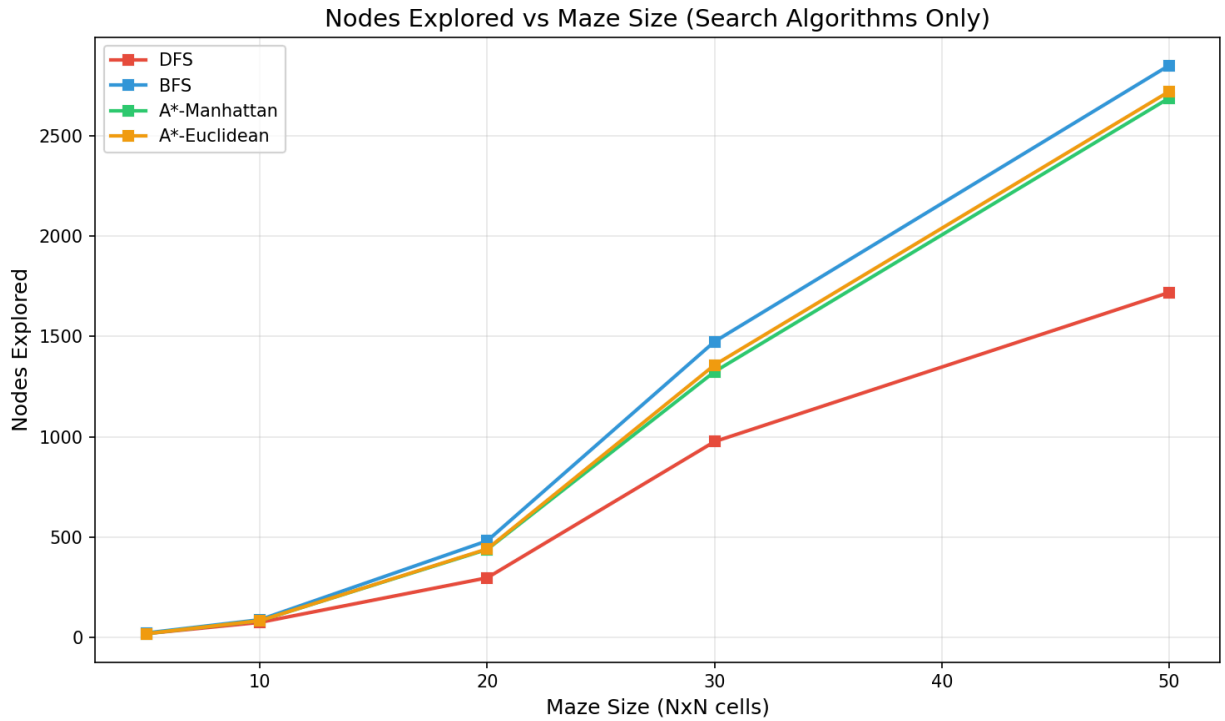

Figure 4: Nodes Explored vs Maze Size (Search Algorithms)

Maximum Frontier Size: DFS exhibited the largest maximum frontier, growing to approximately 29 at 50×50, because its stack stores the entire current path from root to the deepest expanded node. BFS maintained the smallest frontier (≈8), due to the tree like structure of perfect mazes where branching is minimal. A* Manhattan and A* Euclidean showed frontier sizes between BFS and DFS.

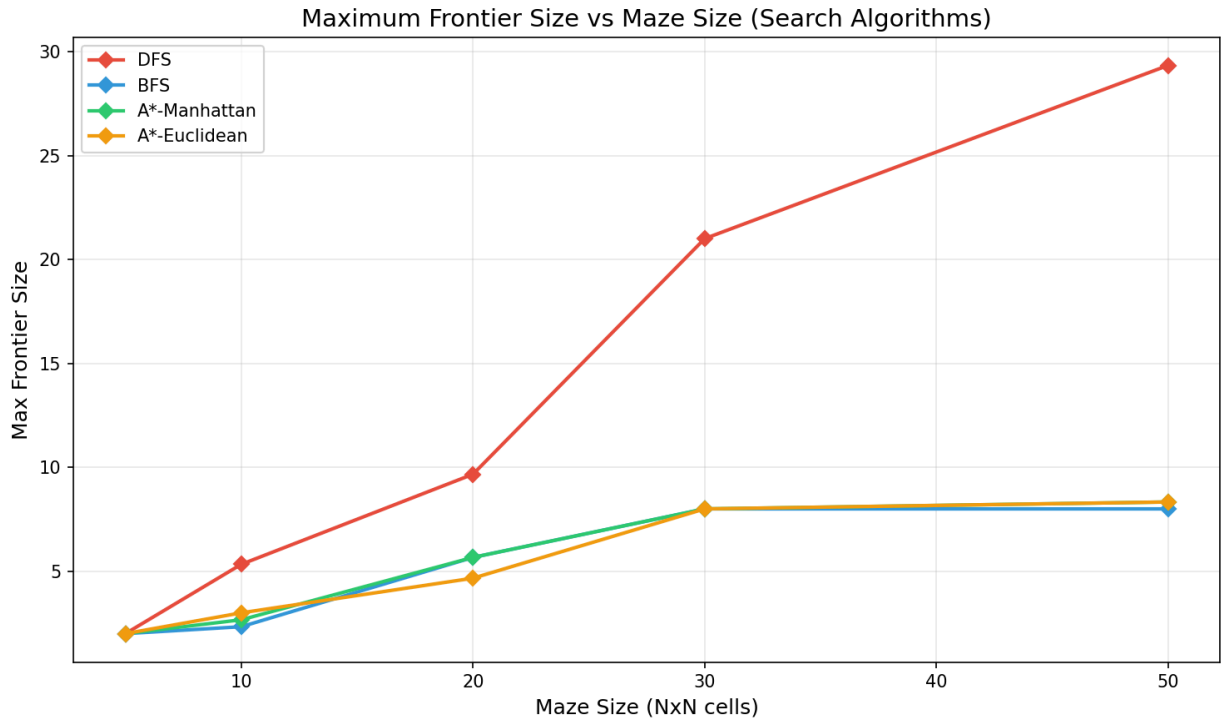

Figure 5: Maximum Frontier Size vs Maze Size

Note: DFS having the largest frontier is counterintuitive for general graphs. In perfect mazes, BFS's frontier is very narrow because branching factor ≈ 1 at each level, while DFS's stack grows proportionally with the depth of the single long corridor.

Execution Time: DFS was the fastest at all sizes (7.6ms at 50×50). BFS followed at 14.0ms. A* Manhattan (18.0ms) and A* Euclidean (17.3ms) were slowest due to the overhead of maintaining a priority queue (heap operations) vs the simple stack (DFS) or deque (BFS).

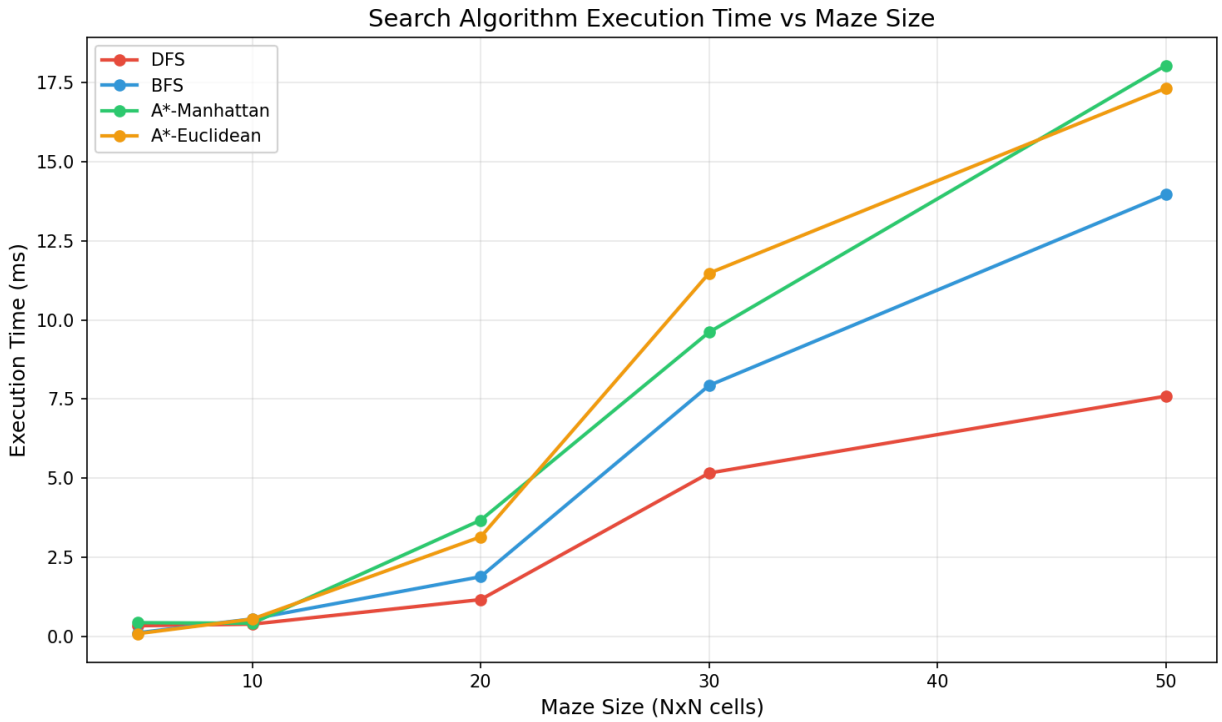

Figure 6: Search Algorithm Execution Time vs Maze Size

MDP Algorithm Comparison

Both MDP algorithms successfully solved all mazes up to 30×30. At 50×50, both achieved 2 out of 3 successful solves, with one run failing to extract a valid path to the goal despite value convergence. Figure 7 shows 20×20 mazes solved by A* Manhattan and Value Iteration, confirming both find identical optimal paths.

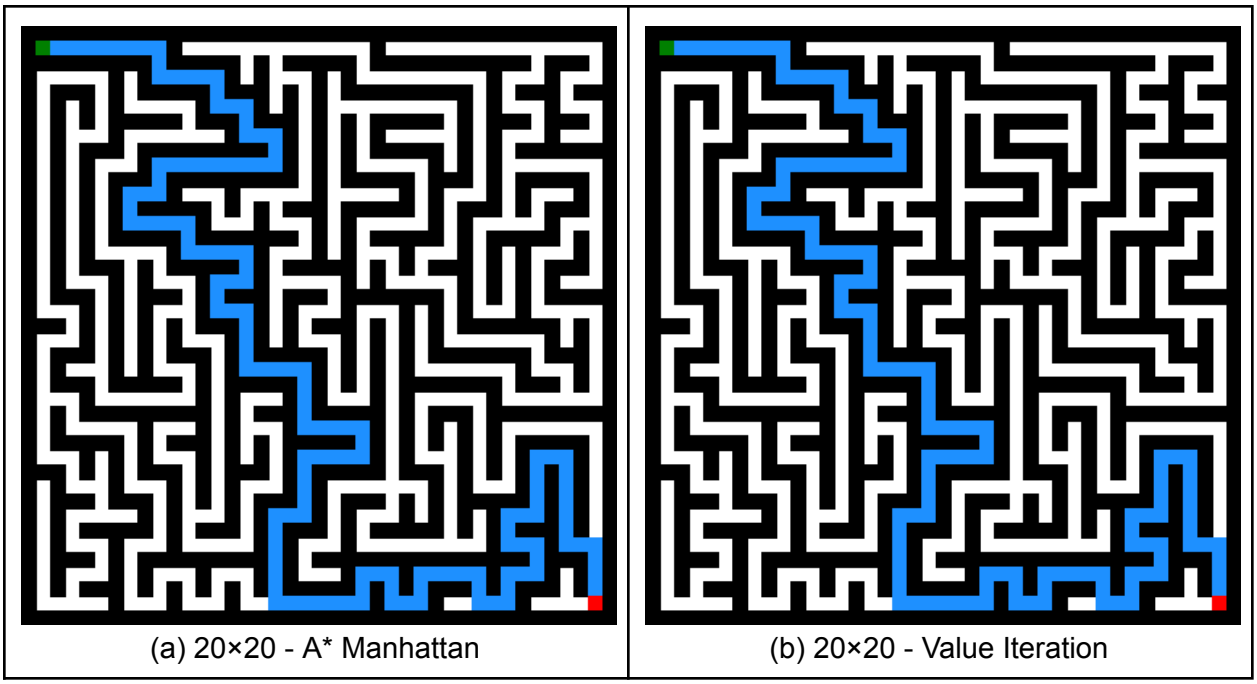

Figure 7: 20×20 mazes solved by search (A Manhattan) and MDP (Value Iteration) - identical paths.*

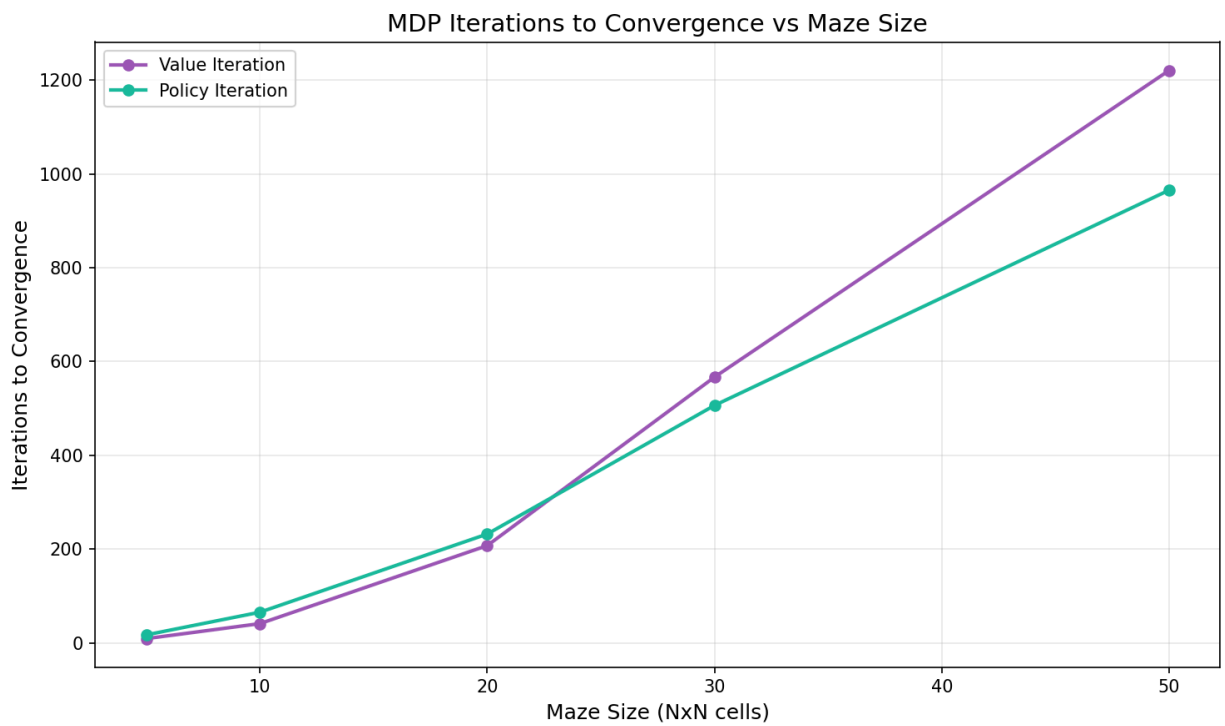

Figure 8: MDP Iterations to Convergence vs Maze Size

Value Iteration required more iterations than Policy Iteration at every size (1,221 vs 965 at 50×50). However, each PI iteration is substantially more expensive due to the policy evaluation

sub procedure. Despite needing fewer iterations, PI performed far more total computational work: ~37.7M vs ~24.4M state action evaluations at 50×50.

Size	VI Iters	PI Iters	VI Time(s)	PI Time(s)	VI Evals	PI Evals
5×5	9	17	0.012	0.261	1,728	72,352
10×10	41	65	0.131	1.352	32,472	360,822
20×20	207	232	4.716	15.927	661,808	2,291,590
30×30	567	506	27.779	54.237	4,075,467	8,202,476
50×50	1,221	965	150.791	247.359	24,403,568	37,729,902

Table 2: MDP Performance Summary

Cross Family Comparison: Search vs MDP

The performance gap between search and MDP algorithms is dramatic and grows with maze size. Figure 9 shows the full comparison on a single chart.

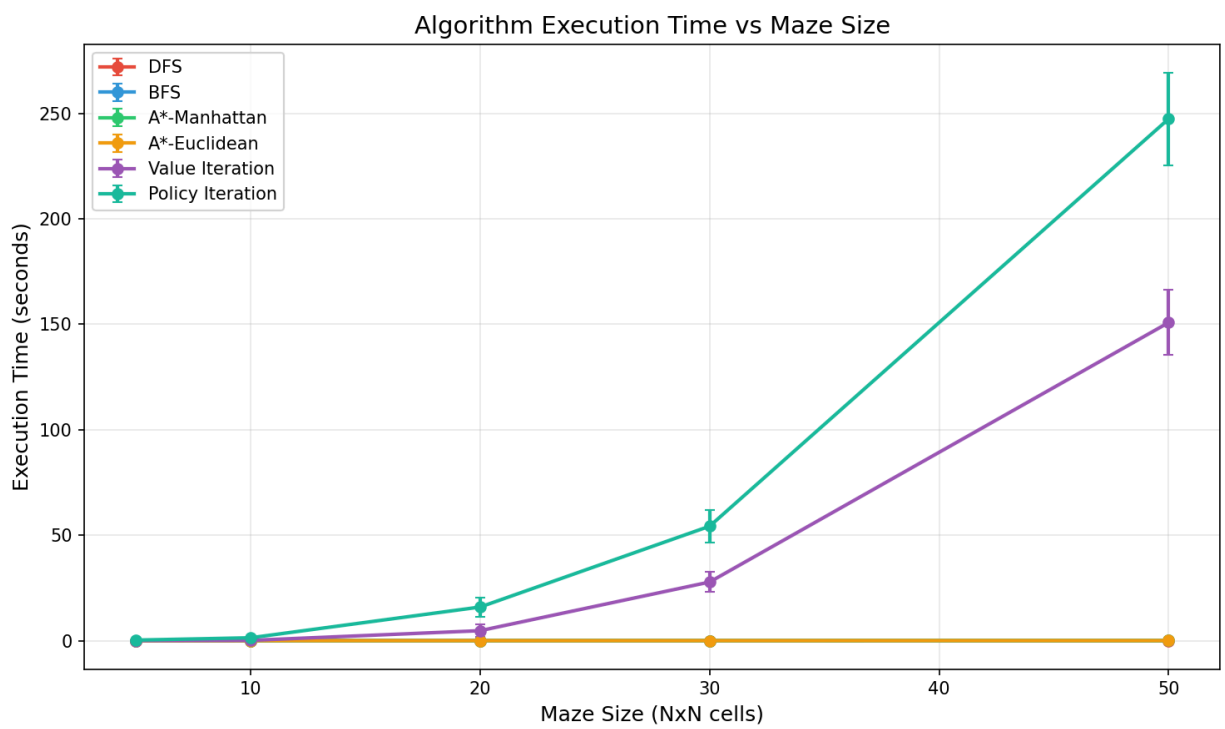

Figure 9: Algorithm Execution Time vs Maze Size (All Algorithms)

At 50×50, DFS (7.6ms) is approximately 19,800 times faster than Value Iteration (150.8s) more than four orders of magnitude. This gap reflects a fundamental architectural difference: search

algorithms explore only solution relevant states, while MDP algorithms compute policies for the entire state space.

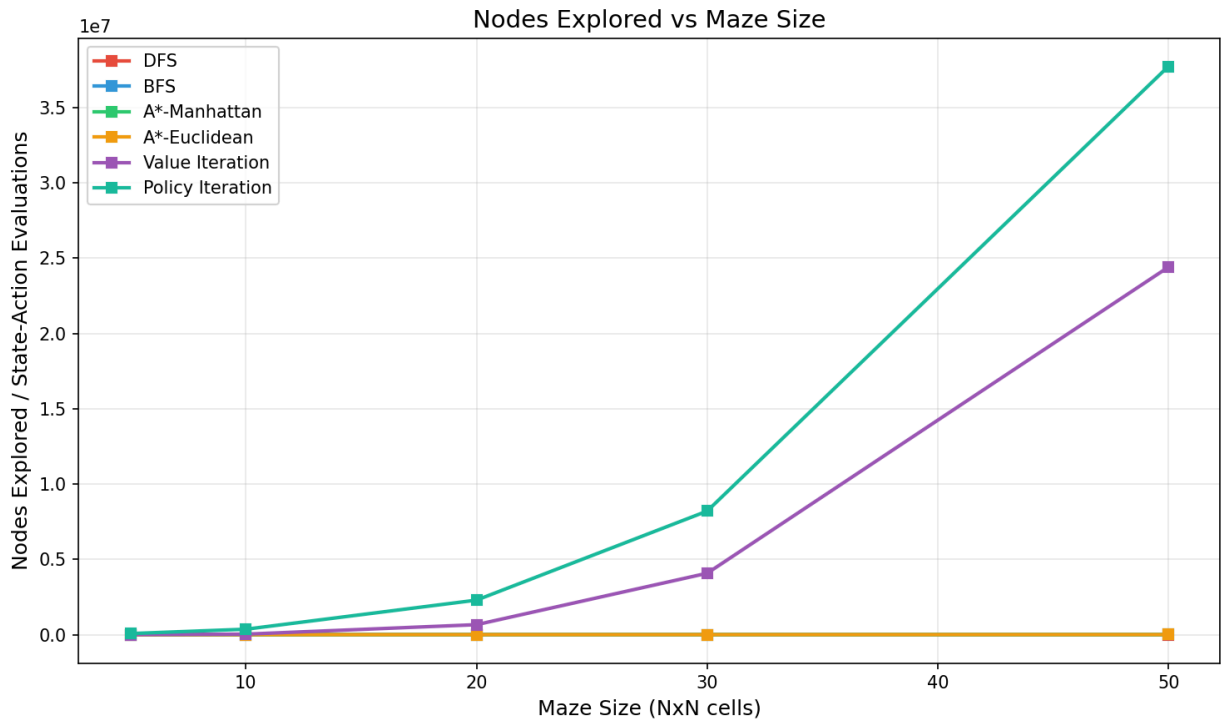

Figure 10: Nodes Explored vs Maze Size (All Algorithms)

DFS explored 1,718 nodes at 50×50 while Value Iteration performed 24.4 million state action evaluations a ratio of 14,200:1. In a deterministic maze with known structure, the MDP formulation provides no advantage.

Figure 11 shows the scaling contrast at 30×30 and 50×50, where MDP solution paths are visually identical to search paths but require orders of magnitude more computation.

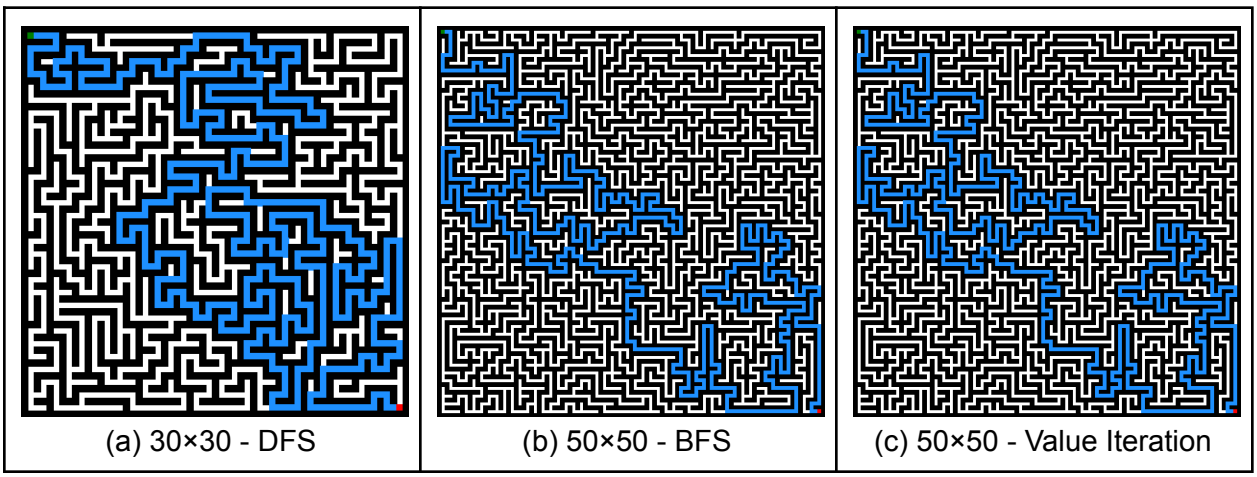

Figure 11: Solved mazes at larger sizes. Search and MDP find identical paths despite vastly different computational costs.

MDP Parameter Sensitivity Analysis

Impact of Discount Factor (γ)

The discount factor γ determines how much the agent values future rewards relative to immediate rewards. Three values were tested: γ ∈ {0.9, 0.95, 0.99}, on maze sizes {5, 10, 20} with 3 runs each

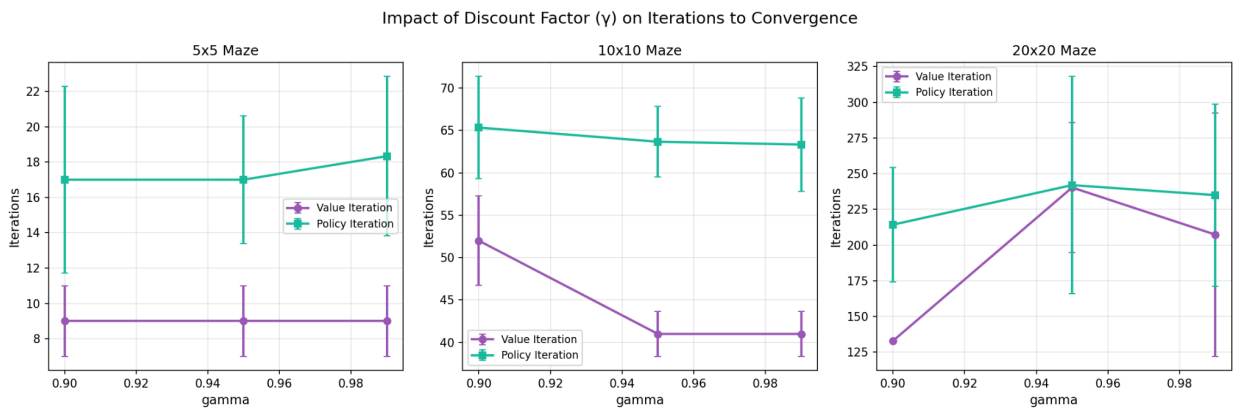

Figure 12: Impact of Discount Factor (γ) on Iterations to Convergence

For Value Iteration, a counterintuitive pattern emerges at 10×10: higher γ (0.99) required fewer iterations (41) than lower γ (0.9, requiring 52). Lower γ discounts future rewards more aggressively, making the distant goal reward appear less significant, so more iterations are needed for value differences to propagate correctly. At 20×20, the pattern becomes more complex due to maze topology effects, with γ=0.95 taking the most iterations (240 mean) while γ=0.9 took 133 and γ=0.99 took 207.

For Policy Iteration, iteration counts were relatively stable across γ values at small sizes (5×5 and 10×10). At 20×20, PI showed more variation between runs than between γ values.

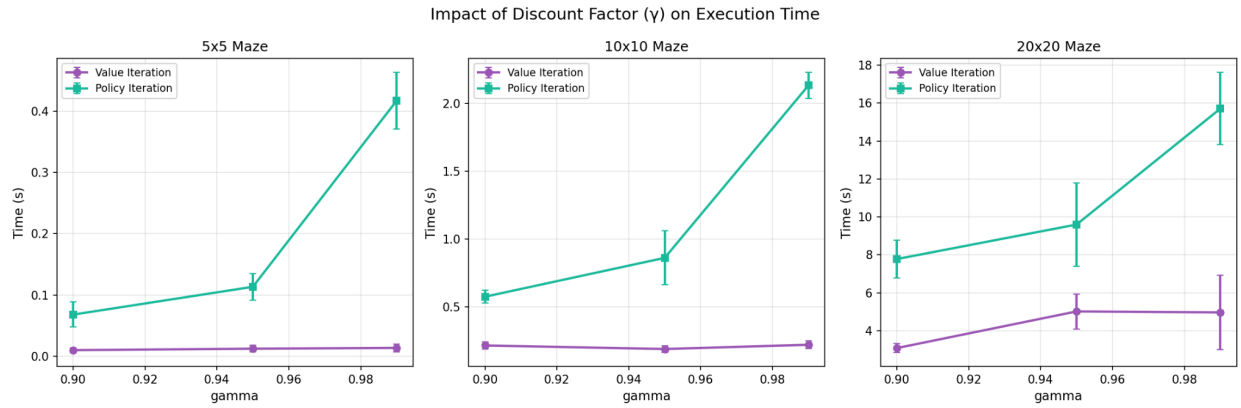

Figure 13: Impact of Discount Factor (γ) on Execution Time

Execution time shows a clear trend for Policy Iteration: higher γ leads to substantially longer times. At 5×5, PI took 0.07s with γ=0.9 but 0.42s with γ=0.99 a 6x increase. At 20×20, this grew to 7.8s vs 15.7s. Higher γ makes the policy evaluation sub step harder to converge because the effective planning horizon is longer. Value Iteration's time was comparatively stable across γ values.

γ=0.9 caused path-finding failures at 20×20 (2 out of 3 VI runs failed) because heavy discounting attenuated the distant goal reward too much. γ=0.99 was the most reliable setting.

Impact of Transition Stochasticity (p_intended)

The p_intended parameter controls whether the agent moves deterministically or stochastically. Four values were tested: p_intended ∈ {0.7, 0.8, 0.9, 1.0}.

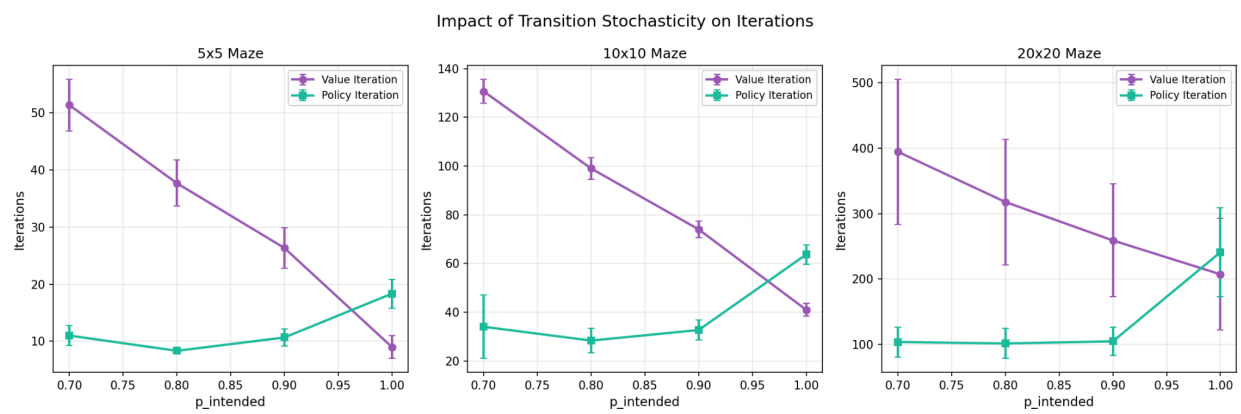

Figure 14: Impact of Transition Stochasticity on Iterations

The results reveal a striking divergence. For Value Iteration, lower p_intended dramatically increased iterations: at 10×10, VI needed 131 iterations with p=0.7 versus 41 with p=1.0 (3.2x increase). Stochastic transitions create a more complex value landscape requiring more sweeps. For Policy Iteration, the opposite holds: lower p_intended actually reduced outer iterations (34 at p=0.7 vs 64 at p=1.0 for 10×10). Stochastic transitions make policies simpler

more policies are equally good so PI converges to a stable policy faster. However, each iteration is far more expensive

Figure 15: Impact of Transition Stochasticity on Execution Time

For PI at 20×20, p=0.7 took ~96s versus ~15s for p=1.0 a 6.4x slowdown. The stochastic transition function means each state-action evaluation must consider multiple possible successor states. VI showed a milder increase (9.4s vs 4.7s at 20×20).

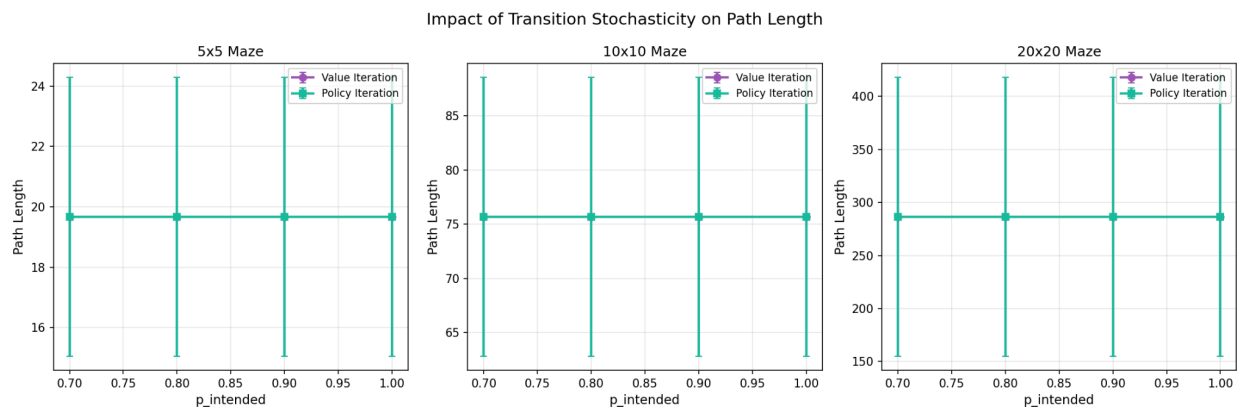

Figure 16: Impact of Transition Stochasticity on Path Length

Path lengths were identical across all p_intended values. Stochastic transitions affect policy computation but not the extracted path, which follows the greedy policy deterministically from start to goal.

Discussion

The experimental results clearly demonstrate that search algorithms are vastly superior to MDP algorithms for deterministic maze solving. The four order of magnitude time gap at 50×50 reflects a fundamental architectural mismatch. MDP algorithms compute policies for the entire state space, which is computationally wasteful when only a single start to goal path is needed and when the environment is fully observable and deterministic.

However, MDP algorithms have advantages in domains where their formulation is appropriate. The stochasticity experiments (Section 5.2) demonstrate this: when transitions are uncertain (p_intended < 1.0), MDP algorithms naturally account for the probability of unintended movements and produce policies robust to noise. A search algorithm would need to be run repeatedly or combined with re-planning strategies to handle stochastic environments, whereas MDPs handle uncertainty natively within a single computation.

Among search algorithms, DFS is the most efficient for perfect mazes due to the unique path property: its tendency to go deep before backtracking leads it directly to the solution without unnecessary exploration. BFS explores more nodes due to systematic level by level expansion. A* provides no significant advantage in perfect mazes because the heuristic cannot effectively prune the search space when only one path exists.

Among MDP algorithms, Value Iteration proved more efficient across all configurations. PI's theoretical advantage of convergence in fewer iterations is overwhelmed by the per iteration cost, particularly at γ=0.99 where policy evaluation itself requires many sub iterations to converge.

The γ sensitivity analysis reveals an important practical trade off: γ=0.99 provides the most reliable convergence (fewest failures) but at the cost of longer execution times. Lower γ values (0.9) are faster but occasionally fail to propagate the goal reward across large mazes.

Limitations of this analysis include:

1. only perfect mazes were tested mazes with loops would change relative performance of DFS vs BFS and A*
2. the stochastic experiments used the same perfect maze topology
3. the implementation is in pure Python without optimisation, so absolute timings should be interpreted with caution.

Conclusion

This report compared four search algorithms (DFS, BFS, A* Manhattan, A* Euclidean) and two MDP algorithms (Value Iteration, Policy Iteration) for maze solving across five maze sizes. The key findings are:

- Search algorithms are orders of magnitude faster than MDP algorithms for deterministic maze solving, with DFS being the most efficient overall due to the unique path property of perfect mazes. At 50×50, DFS solved mazes in 7.6ms while Value Iteration required 150.8 seconds.
- All algorithms found identical path lengths on perfect mazes, confirming both the unique solution property and the correctness of all implementations.
- Value Iteration outperformed Policy Iteration in total computational work and execution time at every maze size, despite PI requiring fewer outer iterations.
- The discount factor γ=0.99 provides the most reliable convergence but incurs longer execution times, particularly for Policy Iteration. Lower γ values risk convergence failures on larger mazes.

- Stochastic transitions (p_intended < 1.0) substantially increase MDP computation time, with Policy Iteration particularly affected. However, MDP algorithms handle stochastic environments natively, an advantage that search algorithms lack.

These results align with theoretical expectations from Russell and Norvig (2021): search algorithms are preferred for deterministic, fully observable problems, while MDP algorithms are suited for sequential decision making under uncertainty.

References

[1] Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th Ed. Pearson.

[2] Bellman, R. (1957) Dynamic Programming. Princeton University Press.

[3] Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', IEEE Trans. Systems Science and Cybernetics, 4(2), pp. 100–107.

[4] Buck, J. (2015) Mazes for Programmers. Pragmatic Bookshelf.

[5] Sutton, R.S. and Barto, A.G. (2018) Reinforcement Learning: An Introduction, 2nd Ed. MIT Press.

[6] CS7IS2 Artificial Intelligence Lecture Notes (2026), Trinity College Dublin

Appendices

Appendix 1: DFS Implementation

```python
def dfs(maze: Maze) -> SearchResult:
    """
    Solve the maze using Depth-First Search.

    Uses an explicit stack (iterative) to avoid Python recursion limits
    on large mazes.

    Properties:
        - Not guaranteed to find the shortest path.
        - Memory efficient — O(longest path) frontier size.
        - Time: O(V + E) in the worst case.
    """
    start_time = time.perf_counter()
```

```
    start, goal = maze.start, maze.goal
    stack = [start]
    visited = {start}
    parents = {start: None}
    nodes_explored = 0
    max_frontier = 1

    while stack:
        max_frontier = max(max_frontier, len(stack))
        current = stack.pop()
        nodes_explored += 1

        if current == goal:
            elapsed = time.perf_counter() - start_time
            path = _reconstruct_path(parents, start, goal)
            return SearchResult("DFS", path, nodes_explored, max_frontier,
elapsed)

        for neighbour in maze.get_neighbours(*current):
            if neighbour not in visited:
                visited.add(neighbour)
                parents[neighbour] = current
                stack.append(neighbour)

    elapsed = time.perf_counter() - start_time
    return SearchResult("DFS", None, nodes_explored, max_frontier,
elapsed)
```

Appendix 2: BFS Implementation

```
def bfs(maze: Maze) -> SearchResult:
    """
    Solve the maze using Breadth-First Search.

    Uses a FIFO queue (collections.deque).

    Properties:
        - Guarantees the shortest path (in terms of number of steps).
        - Higher memory usage than DFS — O(branching_factor^depth).
        - Time: O(V + E).
    """
```

```
    from collections import deque
    start_time = time.perf_counter()

    start, goal = maze.start, maze.goal
    queue = deque([start])
    visited = {start}
    parents = {start: None}
    nodes_explored = 0
    max_frontier = 1

    while queue:
        max_frontier = max(max_frontier, len(queue))
        current = queue.popleft()
        nodes_explored += 1

        if current == goal:
            elapsed = time.perf_counter() - start_time
            path = _reconstruct_path(parents, start, goal)
            return SearchResult("BFS", path, nodes_explored, max_frontier,
elapsed)

        for neighbour in maze.get_neighbours(*current):
            if neighbour not in visited:
                visited.add(neighbour)
                parents[neighbour] = current
                queue.append(neighbour)

    elapsed = time.perf_counter() - start_time
    return SearchResult("BFS", None, nodes_explored, max_frontier,
elapsed)
```

Appendix 3: A* Search Implementation (Manhattan)

```
def _manhattan_heuristic(node: Tuple[int, int], goal: Tuple[int, int]) ->
int:
    """Manhattan (L1) distance — admissible for grid movement."""
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])


def astar_manhattan(maze: Maze) -> SearchResult:
    """
```

```
    Solve the maze using A* with Manhattan distance heuristic.

    f(n) = g(n) + h(n) where h = Manhattan distance to goal.

    Properties:
        - Optimal and complete (Manhattan is admissible + consistent).
        - Generally explores fewer nodes than BFS.
    """
    start_time = time.perf_counter()

    start, goal = maze.start, maze.goal
    # Priority queue entries: (f_score, tie_breaker, node)
    counter = 0
    open_set = [(0 + _manhattan_heuristic(start, goal), counter, start)]
    g_score = {start: 0}
    parents = {start: None}
    closed_set = set()
    nodes_explored = 0
    max_frontier = 1

    while open_set:
        max_frontier = max(max_frontier, len(open_set))
        _, _, current = heapq.heappop(open_set)

        if current in closed_set:
            continue

        closed_set.add(current)
        nodes_explored += 1

        if current == goal:
            elapsed = time.perf_counter() - start_time
            path = _reconstruct_path(parents, start, goal)
            return SearchResult("A*-Manhattan", path, nodes_explored,
max_frontier, elapsed)

        for neighbour in maze.get_neighbours(*current):
            if neighbour in closed_set:
                continue
            tentative_g = g_score[current] + 1  # uniform cost per step

            if tentative_g < g_score.get(neighbour, float('inf')):
```

```
                g_score[neighbour] = tentative_g
                f = tentative_g + _manhattan_heuristic(neighbour, goal)
                parents[neighbour] = current
                counter += 1
                heapq.heappush(open_set, (f, counter, neighbour))

    elapsed = time.perf_counter() - start_time
    return SearchResult("A*-Manhattan", None, nodes_explored,
max_frontier, elapsed)

def _euclidean_heuristic(node: Tuple[int, int], goal: Tuple[int, int]) ->
float:
    """Euclidean (L2) distance — admissible but less informed than
Manhattan for grids."""
    return math.sqrt((node[0] - goal[0]) ** 2 + (node[1] - goal[1]) ** 2)
```

Appendix 3b: A* Search Implementation (Euclidean)

```
def astar_euclidean(maze: Maze) -> SearchResult:
    """
    Solve the maze using A* with Euclidean distance heuristic.

    f(n) = g(n) + h(n) where h = Euclidean distance to goal.

    Properties:
        - Optimal and complete (Euclidean is admissible + consistent).
        - Less informed than Manhattan on 4-connected grids, so typically
          explores more nodes than A*-Manhattan.
    """
    start_time = time.perf_counter()

    start, goal = maze.start, maze.goal
    counter = 0
    open_set = [(0 + _euclidean_heuristic(start, goal), counter, start)]
    g_score = {start: 0}
    parents = {start: None}
    closed_set = set()
    nodes_explored = 0
    max_frontier = 1

    while open_set:
```

```
        max_frontier = max(max_frontier, len(open_set))
        _, _, current = heapq.heappop(open_set)

        if current in closed_set:
            continue

        closed_set.add(current)
        nodes_explored += 1

        if current == goal:
            elapsed = time.perf_counter() - start_time
            path = _reconstruct_path(parents, start, goal)
            return SearchResult("A*-Euclidean", path, nodes_explored,
max_frontier, elapsed)

        for neighbour in maze.get_neighbours(*current):
            if neighbour in closed_set:
                continue
            tentative_g = g_score[current] + 1

            if tentative_g < g_score.get(neighbour, float('inf')):
                g_score[neighbour] = tentative_g
                f = tentative_g + _euclidean_heuristic(neighbour, goal)
                parents[neighbour] = current
                counter += 1
                heapq.heappush(open_set, (f, counter, neighbour))

    elapsed = time.perf_counter() - start_time
    return SearchResult("A*-Euclidean", None, nodes_explored,
max_frontier, elapsed)
```

Appendix 4: Value Iteration Implementation

```
def value_iteration(maze: Maze, gamma: float = 0.99,
                    theta: float = 1e-6, max_iterations: int = 5000,
                    p_intended: float = 1.0,
                    step_reward: float = -1.0,
                    goal_reward: float = 100.0) -> MDPResult:
    """
    Solve the maze MDP using Value Iteration.
```

```
Repeatedly applies the Bellman optimality update:
    V(s) = max_a  Σ_s' T(s,a,s') [ R(s,a,s') + γ V(s') ]
until convergence (max change < theta).

Parameters
----------
maze : Maze
    The maze to solve.
gamma : float
    Discount factor (0 < gamma <= 1).
theta : float
    Convergence threshold.
max_iterations : int
    Safety limit on iterations.
p_intended : float
    Probability of moving in the intended direction.
step_reward : float
    Reward (penalty) for each step taken.
goal_reward : float
    Reward for reaching the goal.

Returns
-------
MDPResult
"""
start_time = time.perf_counter()

# Identify all open (passable) states
open_cells = maze.get_all_open_cells()
states = set(open_cells)

# Initialise value function
V = {s: 0.0 for s in states}
V[maze.goal] = goal_reward  # terminal state

nodes_explored = 0  # total state-action evaluations
converged = False

for iteration in range(1, max_iterations + 1):
    delta = 0.0
```

```
        for s in states:
            if s == maze.goal:
                continue  # goal is terminal

            old_v = V[s]
            best_value = float('-inf')

            for action in ACTION_LIST:
                action_value = 0.0
                transitions = _get_transitions(maze, s, action,
p_intended)

                for prob, next_state in transitions:
                    if next_state in states:
                        reward = goal_reward if next_state == maze.goal
else step_reward
                        action_value += prob * (reward + gamma *
V[next_state])
                    else:
                        # Hit a wall — stay in place with penalty
                        action_value += prob * (step_reward + gamma *
V[s])

                nodes_explored += 1
                best_value = max(best_value, action_value)

            V[s] = best_value
            delta = max(delta, abs(old_v - V[s]))

        if delta < theta:
            converged = True
            break

    # Extract policy from converged value function
    policy = {}
    for s in states:
        if s == maze.goal:
            continue
        best_action = None
        best_value = float('-inf')
        for action in ACTION_LIST:
            action_value = 0.0
```

```
            transitions = _get_transitions(maze, s, action, p_intended)
            for prob, next_state in transitions:
                if next_state in states:
                    reward = goal_reward if next_state == maze.goal else
step_reward
                    action_value += prob * (reward + gamma *
V[next_state])
                else:
                    action_value += prob * (step_reward + gamma * V[s])
            if action_value > best_value:
                best_value = action_value
                best_action = action
        policy[s] = best_action

    path = _extract_path(maze, policy)
    elapsed = time.perf_counter() - start_time

    return MDPResult("Value Iteration", path, nodes_explored, iteration,
                     elapsed, converged)
```

Appendix 5: Policy Iteration Implementation

```
def policy_iteration(maze: Maze, gamma: float = 0.99,
                     theta: float = 1e-6, max_iterations: int = 1000,
                     p_intended: float = 1.0,
                     step_reward: float = -1.0,
                     goal_reward: float = 100.0) -> MDPResult:
    """
    Solve the maze MDP using Policy Iteration.

    Alternates between:
      1. Policy Evaluation — compute V^π for the current policy π.
      2. Policy Improvement — greedily update π w.r.t. V^π.
    Converges when the policy is stable (no changes in an improvement
step).

    Parameters
    ----------
    (same as value_iteration)

    Returns
```

```
    -------
    MDPResult
    """
    start_time = time.perf_counter()

    open_cells = maze.get_all_open_cells()
    states = set(open_cells)
    states_list = list(states - {maze.goal})

    # Initialise random policy
    import random
    policy = {s: random.choice(ACTION_LIST) for s in states_list}

    V = {s: 0.0 for s in states}
    V[maze.goal] = goal_reward

    nodes_explored = 0
    converged = False

    for iteration in range(1, max_iterations + 1):
        # ---- Policy Evaluation ----
        while True:
            delta = 0.0
            for s in states_list:
                old_v = V[s]
                action = policy[s]
                action_value = 0.0
                transitions = _get_transitions(maze, s, action,
p_intended)

                for prob, next_state in transitions:
                    if next_state in states:
                        reward = goal_reward if next_state == maze.goal
else step_reward
                        action_value += prob * (reward + gamma *
V[next_state])
                    else:
                        action_value += prob * (step_reward + gamma *
V[s])

                nodes_explored += 1
                V[s] = action_value
```

```
                delta = max(delta, abs(old_v - V[s]))

            if delta < theta:
                break

        # ---- Policy Improvement ----
        policy_stable = True
        for s in states_list:
            old_action = policy[s]
            best_action = None
            best_value = float('-inf')

            for action in ACTION_LIST:
                action_value = 0.0
                transitions = _get_transitions(maze, s, action,
p_intended)
                for prob, next_state in transitions:
                    if next_state in states:
                        reward = goal_reward if next_state == maze.goal
else step_reward
                        action_value += prob * (reward + gamma *
V[next_state])
                    else:
                        action_value += prob * (step_reward + gamma *
V[s])
                nodes_explored += 1
                if action_value > best_value:
                    best_value = action_value
                    best_action = action

            policy[s] = best_action
            if old_action != best_action:
                policy_stable = False

        if policy_stable:
            converged = True
            break

    path = _extract_path(maze, policy)
    elapsed = time.perf_counter() - start_time

    return MDPResult("Policy Iteration", path, nodes_explored, iteration,
```

```
                    elapsed, converged)
```